

VASTRANGAM

MEDHAVA — PLAN OF ACTION

One business operating system. Any industry. One shared data core.

22 modules · 112 apps · 277 rules · 113 tables · 46 product screens across 12 sectors. Every count in this document is read from the canonical files when it is generated — the module list, the rulebook, the schema and the tools register — so no figure here was typed from memory.

What this document is. The plan for building Medhava as a product that any business can adopt, from planning through execution. It is not the Vastrangam plan with the names changed. `PLAN_OF_ACTION.md` is that: one ethnic-wear group in Surat adopting this engine, with its own karigar payroll, its own three companies, its own migration off an accounting package. That document stays as it is, and it is the most useful thing in this repository — it is the worked example of the whole exercise, an entire industry implementation carried to the level of detail that proves the engine bends.

This document answers the different question: **how does the same engine serve a dental clinic and a freight forwarder and a dairy co-operative, without becoming a different program each time?**

PART I — WHAT MEDHAVA IS

M1 · ONE ENGINE, MANY TRADES

Medhava is a single application over one shared data core. A business event is entered once and flows everywhere it belongs — that law, and the cascades that follow from it, are set out in full in `PLAN_OF_ACTION.md` §A0 and are identical here, because it is the same engine. This document does not restate them.

What is specific to Medhava is the claim that the engine does not know which trade it is in. That claim is either true in the code or it is marketing, so here is where it is enforced:

The claim	What makes it true
No module assumes an industry	<code>brand/site/modules.js</code> is audited at build time; trade vocabulary in it fails the build
Trade words live in one place	The edition overlay may change wording only — <code>build.js</code> compares the structural shape before and after applying it and exits if a module number, an app name or an app count moved
The same structure ships twice already	Two editions build from one module list: MEDHAVA neutral, VASTRANGAM in one trade's own words
The number of companies is data	<code>core/tests/core.test.js</code> posts across a 10 × 10 company × channel grid, then runs 11 × 11 with no code changed

The sentence the whole product rests on. A clinic's appointment, a factory's work order, a law firm's matter, a contractor's site and a service firm's job are the same record with different words on it. Everything below is the work of making that true rather than clever.

M2 · THE THING THAT IS NOT YET BUILT, NAMED FIRST

Two editions is not "any industry". Two editions is two editions. A third trade today means someone writing a third overlay file by hand, and that does not scale to a product.

The industry pack is the answer, and it is **specified, not built**. An industry pack is a row of configuration, loaded as data, that carries:

- **vocabulary** — what this trade calls an order, a customer, a unit of work, a stage
- **stages** — the pipeline a job moves through, named and ordered by the trade
- **fields** — the extra attributes this trade needs on a record, and their types
- **documents** — the papers it issues, and what has to be on them
- **rules that switch on** — which of the 277 apply, and which thresholds they use
- **starting data** — a chart of accounts, tax rates, roles, units of measure

The engine reads a pack the way it already reads companies and channels: as rows. Nothing in `modules.js`, `core/` or the schema changes when a fourteenth trade is added — which is exactly the property the 10 × 10 test already proves for companies, applied to trades.

Until the pack engine exists, "every industry" is a design commitment and not a shipped feature, and this document says so in the same breath as it makes the claim. Phase 2 is where it gets built, and it is the phase everything else depends on.

M3 · TENANCY — WHAT IS A ROW AND WHAT IS CODE

Concept	Row or code	Consequence
Tenant (a customer of Medhava)	row	Onboarding a business is data entry, not a deployment
Company inside a tenant	row	A group with four companies is four rows, consolidated with inter-company trade eliminated
Channel	row	A new marketplace is a row; rule R15.1 makes discovery automatic
Location, stage, role	row	A warehouse, a production stage and a job title are all configuration
Industry pack	row (to build)	A trade is configuration, not a fork
The 22 modules	code	The structure is the product; it is the same for everyone
The 277 rules	code	Which apply is configurable; what they refuse is not

Isolation. Every business table carries `company_id` and a row-level security policy carrying both `USING` and `WITH CHECK`, so a read and a write are separately prevented from crossing a boundary. `core/tests/schema.test.js` fails the build if any company-scoped table lacks one. Cross-tenant isolation is the same mechanism one level up and is the single highest-risk item in this plan — a bug there is not a defect, it is an incident.

M4 · THE 22 MODULES, READ FROM TWELVE TRADES

The module list is in `PLAN_OF_ACTION.md` Part II in full, with all 112 apps and all 277 rules. It is not repeated here. What is here is the reading that makes it industry-neutral — the same module, seen from a different trade:

Module	Manufacturing	Professional services	Healthcare	Hospitality
02 Design & Sampling	part revision	matter scoping	protocol	recipe development
05 Sales	order book	engagement letter	appointment book	cover forecast
06 Planning	material requirement	resourcing	rota and stock	purchase plan
08 Manufacturing	work order, stages	—	—	batch and prep
09 Quality	inspection	file review	licence and calibration	food safety log
10 Warehouse	pick and pack	—	consumables	store issue
16 HR	shift and piece-rate	timesheets, chargeable	sessions and locums	rota and casuals
20 Projects	improvement work	matters	care pathways	openings

The landing page carries **46 screens across 12 sectors** doing exactly this — the same module rendered with a machine shop's figures and a clinic's figures, one under the other. That is the argument made where it can be checked rather than believed.

PART II — EXECUTION

M5 · THE STACK, AND WHAT IT COSTS

The full tools register is `brand/site/tools.js`, gated by `brand/site/checktools.js`, which fails the build if any paid tool does not name **both** the free option it replaces and the concrete condition that forces the upgrade. "When we grow" is rejected by the checker; a number or a named event passes.

19 capabilities. Only three have no free path in existence:

Capability	Why there is no free option	What it costs
WhatsApp Business messaging	Meta requires an approved provider; there is no free tier to outgrow	~₹1,500–3,000/month plus per-conversation charges
SMS	Carrier access is the cost	~₹0.15–0.25 per message
Generated imagery	Free software, but it needs a graphics card	A GPU, or a hosted API per image

Everything else runs on free tiers or free software until a stated trigger fires: Postgres, self-hosted automation, a free hosting tier, local AI, the browser's own speech synthesis, ffmpeg and Chromium for media, UPI for payments, CSV for every integration, and a progressive web app instead of a store listing. **A business can run the whole system on zero rupees of software licence** and pay only for the three lines above, and only when it reaches them.

The Provider Router in Module 01 puts a spend ceiling in front of every paid call and refuses past it rather than warning — so the AI line in particular cannot quietly become the largest one.

M6 · THE EIGHT PHASES

The gate is absolute: **Phase N+1 does not start until Phase N's tests pass.** A phase is done when its stated result is reproduced, not when its code is written.

Phase	What gets built	Done when
0 • Setup	Environments, CI, monitoring, the schema loaded	A commit deploys and a user logs in
1 • Foundation & tenancy	Tenants, companies, roles, RLS, masters, the SKU/item model	Two tenants exist and neither can read a single row of the other, proved by a test that tries
2 • The industry pack engine	Vocabulary, stages, fields, documents and starting data as rows	A third trade is added without writing code — the test that decides whether this is a product
3 • Core operations	Inventory, procurement, production, quality, warehouse	Three trades run the same operations screens on their own vocabulary
4 • Commerce & channels	Sales all channels, OMS, returns, logistics, settlement	A week of orders across channels, settled and reconciled
5 • Finance	Double-entry, tax, banking, period locks	A month closes: trial balance ties and returns generate from vouchers
6 • The AI layer	Assistant, chatbot, agents, guardrails, content	An assistant answer matches the books; an agent asked to move money stops
7 • Onboarding & self-serve	Sign-up, pack selection, import, go-live	A business onboards itself in a day without a call

Phase 2 is the one that matters. If a new trade still needs a developer after Phase 2, Medhava is a consultancy with software attached, not a product — and the phase gate should hold rather than be argued past.

M7 • ONBOARDING A BUSINESS IN A DAY

For a business with no system to migrate from — which is most of the target market — the sixty-day parallel run in the Vastrangam plan is the wrong shape entirely. The sequence is:

1. **Sign up, pick a trade.** The industry pack loads vocabulary, stages, documents and a chart of accounts. Nothing is blank.
2. **Name the company.** One row. A group adds more rows now or later.
3. **Import what exists** — customers, suppliers, items, opening stock — from a spreadsheet, with a validation report **before** anything commits. Errors are shown as rows to fix, never silently skipped.
4. **Opening balances**, or none at all if the business is starting fresh.
5. **Invite people, set roles.** Permissions are per company per role from the first minute.
6. **Do one real transaction end to end** — one sale, invoiced, stock moved, posted — and click the dashboard figure down to the voucher. That is the go-live test, and it takes a minute.

For a business migrating from an existing system, the discipline in `PLAN_OF_ACTION.md` Part IV E5 applies unchanged: export, clean, load, opening balances, parallel run, and a decommission criterion agreed in advance rather than argued at the end.

M8 · SECURITY AND COMPLIANCE

Row-level security in the database and permission checks in API middleware — one layer is one mistake away from a cross-tenant read. Personal data is exportable and erasable on request, with consent and retention tracked as the two separate clocks they are. Card data never reaches this system; the gateway's own secured field takes it, so there is no scope to protect. The audit trail has no off switch, and R16.22 keeps individual pay and personal attributes out of any document that leaves the building.

M9 · PERFORMANCE

Operation	p95
Page load, cached / uncached	< 1 s / < 3 s
Live stock or availability query	< 500 ms
Document PDF	< 2 s
Channel order pull, per channel	< 60 s
Settlement import, 1,000 lines	< 30 s
Daily profit-and-loss refresh	< 10 s

The availability query is the one that decides whether people trust the system: it runs on every screen showing a quantity or a free slot, and one that takes two seconds is one people stop asking.

M10 · RISK REGISTER

Risk	Likelihood	Impact	Mitigation
Cross-tenant data leak	Low	Critical	RLS with USING and WITH CHECK, middleware checks, a test that actively attempts a cross-tenant read
The industry pack engine slips, and trades get hard-coded instead	High	Critical	Phase 2 gate: a third trade added with no code. Hold the gate
A vertical demands a genuine structural exception	Medium	High	Add it as a module or an app for everyone, never as a branch for one customer
Free tier terms change or a tier disappears	Medium	Medium	Every capability has a self-host route recorded; the register is dated
WhatsApp provider outage	Low	High	Adapter swap; SMS and in-app as fallback
Onboarding needs hand-holding, so it does not scale	High	Medium	The day-one sequence above is a tested path, not a document
AI spend runs away	Medium	Medium	The spend ceiling refuses rather than warns

M11 · SUCCESS METRICS

Measure	Target
A new trade supported without writing code	Yes, from Phase 2 onward — binary, not a percentage
Time for a business to onboard itself	Under a day, unattended
Cost to run for a business at pilot scale	₹0 in software licence
Rules enforced by a test	78 of 277 today; up every build
Cross-tenant incidents	Zero, and this is the only metric with no acceptable non-zero value

M12 · RUNBOOKS

Daily — error and uptime check, failed integrations queue, onboarding funnel. **Weekly** — usage per tenant, slow queries, support themes that suggest a missing rule. **Monthly** — free-tier headroom against triggers, spend against ceilings, backup restore test. **Quarterly** — re-read the tools register against current published tiers, review which SPECIFIED rules became enforceable, and check whether any vertical has quietly grown an exception.

M13 · WHAT A CUSTOMER PAYS FOR

The free-first register is not only how Medhava is built — it is the shape of what it can offer. Because the system runs on free tiers and free software at pilot scale, a business can be given a genuinely working system before it pays anything, and the paid lines are the ones it would have had to pay anyway: messaging, marketplace access, payment processing, courier pickup.

The commercial decision — subscription, per-company, per-user, or usage — is not made in this document, because it is a business decision rather than an engineering one. What this document fixes is the constraint it must respect: **nothing in the architecture may make a plan cap technically necessary**. Companies, channels, users and trades are rows. If a plan limits them, that is a commercial choice stated in the pricing, and the software says so rather than pretending to a technical limit it does not have.

PART III — THE PROOF

The test of whether this is one system or twenty-two programs sharing a login is a single transaction followed end to end, and it is re-run at every module boundary. That walkthrough is in `PLAN_OF_ACTION.md` Part V and is identical here.

The test of whether it is *industry-neutral* is different, and it is this: **take the same transaction and run it in a trade nobody had in mind when the code was written**. Until the industry pack engine ships in Phase 2, that test is run by hand by writing an overlay. After Phase 2, it is run by adding a row — and if it still needs a developer, the phase is not done.

Counts in this document are read from `brand/site/modules.js`, `brand/site/rules.js`, `brand/site/shots.js`, `brand/site/tools.js` and `core/schema.postgres.sql` when it is generated. Product screens carry illustrative figures and are labelled with the trade they are drawn from; they are not case studies, and no business is named as a customer that is not one.
